

COLD STORAGE MANAGEMENT SYSTEM

Comprehensive Database Systems Project Report

A Relational Database Approach to Cold Storage Management

Prepared By:

Syed Hisham Azam

Roll Number: 25WU0102282

Course: B.Tech CSE AI/ML

FACULTY OF COMPUTER SCIENCE & ENGINEERING
INSTITUTE OF ADVANCED LOGISTICS & TECHNOLOGY

1 Abstract

Managing cold storage logistics requires strict compliance with temperature limits, precise tracking of inventory batches, and real-time monitoring of sensor data. Standard warehouse platforms often fail to maintain strict thermal audit trails or connect telemetry data with product batch expiration dates. This report presents the design and implementation of the Cold Storage Warehouse Management System (`coldstoredb`), an enterprise-grade relational database solution engineered to streamline cold-chain logistics.

The system uses a MySQL relational database backend connected to a Flask (Python) web framework. The relational schema is normalized up to Third Normal Form (3NF) to eliminate data redundancy, prevent modification anomalies, and enforce referential integrity across multi-warehouse configurations. Key features include active/completed order tracking views, inventory movement logs, automated stock aggregation, and sensor telemetry log ingestion.

To protect sensitive access, user authentication uses `bcrypt` salted hash encryption alongside custom middleware decorators. Dynamic web dashboards provide operational metrics, enabling warehouse managers to track stock balances, audit supplier shipments, and detect temperature anomalies. This document provides a complete technical blueprint of the project, including structural ER diagrams, functional dependency proofs, data dictionaries, SQL DDL/DML scripts, test suites, and source code listings.

Keywords: Cold Storage, Relational Database Management System, MySQL, Flask, Normalization, Data Dictionary, Sensor Telemetry, Inventory Control.

2 Introduction and Problem Statement

2.1 Introduction

Cold-chain management is a specialized branch of supply-chain logistics focused on storing and transporting temperature-sensitive goods—such as dairy, meat, seafood, pharmaceuticals, and fresh produce—within strict thermal ranges. A break in the cold chain can lead to product degradation, financial loss, bacterial growth, or health hazards.

Modern cold storage facilities run complex operations involving multi-zone cooling, supplier inventory intake, quality assurance audits, customer order staging, and continuous sensor logging. Managing these interconnected processes requires a relational database architecture capable of processing transactional records while tracking historical environmental logs.

2.2 Problem Statement

Legacy cold storage warehouses frequently rely on manual logging, isolated desktop spreadsheets, or disconnected software tools. These outdated setups create severe operational bottlenecks:

1. **Thermal Monitoring Gaps:** Environmental sensors capture data independently of inventory records, making it difficult to link a thermal spike directly to affected product batches.
2. **Data Redundancy and Inconsistencies:** Storing customer names, supplier details, and product descriptions across multiple spreadsheets introduces update, insertion, and deletion anomalies.
3. **Inventory Discrepancies:** Manual entries for stock intakes and customer dispatches often cause mismatched inventory counts, leading to stockouts or unrecorded spoilage.
4. **Lack of Real-Time Visibility:** Warehouse personnel lack unified dashboards to monitor current stock levels, pending customer fulfillment, or live sensor alerts simultaneously.
5. **Security and Compliance Audit Deficits:** Plaintext credentials and unaudited data entries fail regulatory compliance standards required for pharmaceutical and food safety handling.

2.3 Proposed Solution

The `coldstoredb` platform addresses these challenges by introducing a unified database architecture:

- **Centralized Relational Core:** Structured MySQL database enforcing foreign key constraints and ACID-compliant transactional consistency.
- **Normalized Schema (3NF):** Eliminates redundancy across suppliers, products, warehouses, orders, and sensors.
- **Automated Aggregate Views:** Dynamic MySQL views generate real-time inventory balances and order staging classifications without manual recalculation.
- **Web-Based Operations Dashboard:** A lightweight Flask interface provides secure access to inventory operations, sensor analytics, and order workflows.

3 Objectives and Scope

3.1 Primary Objectives

The primary goal of this project is to develop a reliable, relational database-backed management application tailored for cold storage facility operations.

1. **Database Architecture Design:** Construct a relational schema normalized to 3NF that represents entities such as users, warehouses, products, suppliers, customers, orders, inventory logs, sensors, and temperature logs.
2. **Integrity Enforcement:** Apply declarative foreign key constraints with cascade/nullify rules to maintain referential integrity across related data records.
3. **Telemetry Data Processing:** Establish structured temperature logging mapped directly to warehouse zones and sensor hardware identifiers.
4. **Secure Web Interface:** Build a web application with Flask that provides session-based access control, bcrypt password hashing, and parameterized SQL queries to prevent SQL injection.
5. **Business Logic & Reporting:** Construct SQL views and analytical queries to deliver operational reporting, such as current stock summaries and active order tracking.

3.2 Scope of the Project

The functional and technical boundaries of the current implementation are defined below:

3.2.1 In-Scope

- **User Authentication:** Secure registration, authentication, session state control, and feedback submission.
- **Warehouse Administration:** Full CRUD (Create, Read, Update, Delete) management of warehouse capacity, location records, and designated thermal operational ranges.
- **Catalog & Supplier Management:** Cataloging products alongside supplier associations, price metrics, and required temperature ranges.
- **Inventory Logging:** Recording positive (intake) and negative (dispatch) stock balance changes with timestamp logs.
- **Order Processing:** Tracking customer purchase orders from pending status through fulfillment state updates.
- **Sensor Monitoring:** Storing sensor records and filtering environmental logs by threshold criteria.

3.2.2 Out-of-Scope

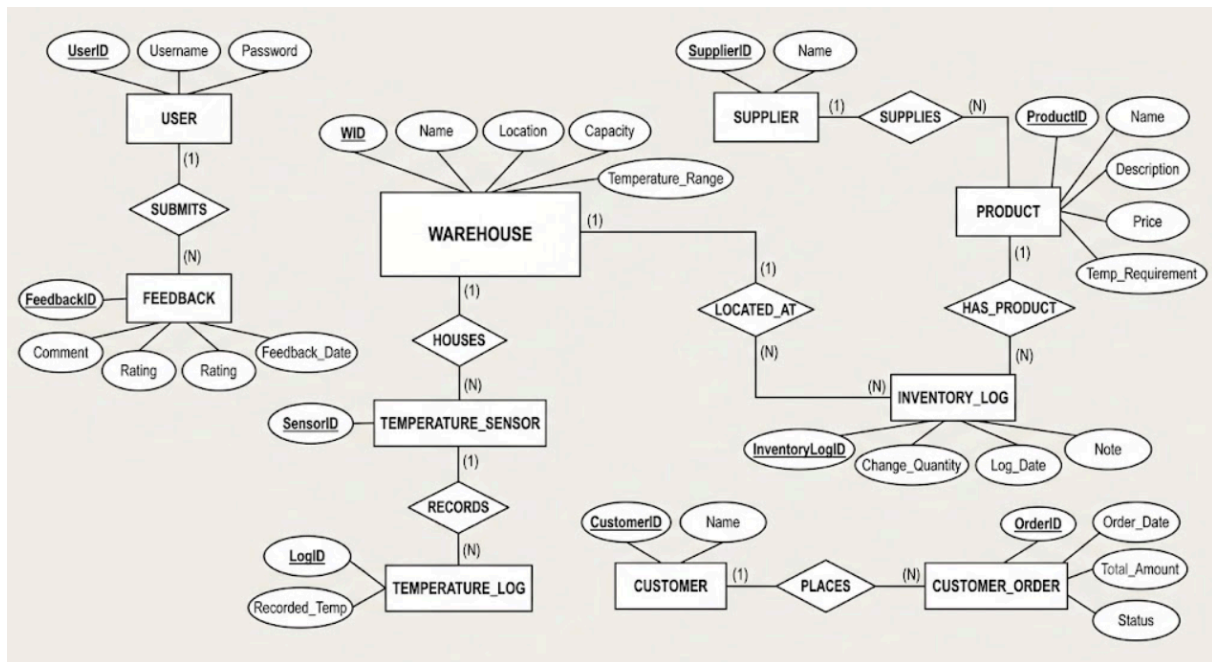
- Direct IoT hardware integration (simulated through database ingestion script endpoints).
- Integrated payment gateways and automated freight invoicing systems.

4 Software and Hardware Requirements

4.1 Environment Specifications

Layer	Component	Specification / Rationale
Database Engine	MySQL Server 8.0	Relational database management system supporting ACID compliance, foreign keys, views, and window functions.
Backend Language	Python 3.10+	High-level language offering robust database connectors and Web Server Gateway Interface (WSGI) integration.
Web Framework	Flask 3.0	Micro-framework providing routing, session handling, and template rendering without unnecessary overhead.
Database Connector	mysql-connector-python	Official MySQL driver enabling parameterized execution to prevent SQL injection.
Security Suite	bcrypt Python Library	Industrial-grade adaptive key derivation function for secure password hashing.
Frontend UI	HTML5, CSS3, JS	Responsive grid structure and fetch API handling for smooth backend communication.
Version Control	Git & GitHub	Source code tracking, schema versioning, and collaborative repository hosting.

5 Entity-Relationship (ER) Diagram



5.1 Entity Definitions and Cardinalities

1. **USER & FEEDBACK:** A User can submit multiple Feedback entries (1 : N). Each feedback record belongs to exactly one User.
2. **SUPPLIER & PRODUCT:** A Supplier supplies one or many Products (1 : N). A Product is linked to a single primary Supplier.
3. **CUSTOMER & CUSTOMER_ORDER:** A Customer places multiple Customer Orders over time (1 : N).
4. **PRODUCT, WAREHOUSE & INVENTORY_LOG:** Products and Warehouses connect through the INVENTORY_LOG relationship. A single Product can have multiple inventory change logs across different Warehouses (1 : N).
5. **WAREHOUSE, SENSOR & TEMPERATURE_LOG:** A Warehouse houses multiple Temperature Sensors (1 : N). Each Temperature Sensor records multiple time-stamped Temperature Logs (1 : N).

6 Relational Schema and Normalization

6.1 Relational Schema Mapping

The conceptual ER model maps directly to the following relational schema:

- user (UserID, Username, Password)
- warehouse (WID, Name, Location, Capacity, Temperature_Range)
- supplier (SupplierID, Name)
- customer (CustomerID, Name)
- product (ProductID, Name, Description, Price, Temp_Requirement, *SupplierID*)
- customer_order (OrderID, *CustomerID*, Order_Date, Total_Amount, Status)
- inventorylog (InventoryLogID, *ProductID*, *WarehouseID*, Change_Quantity, Log_Date, Note)
- temperature_sensor (SensorID, *WarehouseID*)
- temperature_log (LogID, *SensorID*, Recorded_Temp, Timestamp)
- feedback (FeedbackID, *UserID*, Comment, Rating, Feedback_Date)

6.2 Normalization Proofs

6.2.1 First Normal Form (1NF)

A relation is in 1NF if all attribute values are atomic and contain no repeating groups.

- **Analysis:** Every field in the schema (e.g., Price, Recorded_Temp, Capacity) holds a single, indivisible scalar value. Comma-separated list structures were eliminated by breaking inventory operations into separate log rows inside inventorylog.

6.2.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the primary key (no partial dependencies).

- **Analysis:** All tables use single-attribute surrogate primary keys (UserID, WID, ProductID, etc.). Because no primary key is composite, partial functional dependencies are mathematically impossible. Thus, the database satisfies 2NF.

6.2.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key ($X \rightarrow Y$ and $Y \rightarrow Z$).

- **Analysis:** Non-key attributes depend strictly on their respective primary key. For instance, supplier names are not stored in the product table (which would create $\text{ProductID} \rightarrow \text{SupplierID} \rightarrow \text{SupplierName}$). Instead, supplier details are isolated inside supplier, eliminating transitive dependencies and ensuring 3NF compliance.

7 Data Dictionary

The data dictionary below details the table structures, data types, constraints, and descriptions across the database.

7.1 Core Entities

7.1.1 Table: user

Field	Type	Constraint	Description
UserID	INT	PRIMARY KEY, AUTO_INC	Unique system user account ID
Username	VARCHAR(50)	NOT NULL, UNIQUE	Unique login username handle
Password	VAR- CHAR(255)	NOT NULL	Bcrypt salted password hash

7.1.2 Table: warehouse

Field	Type	Constraint	Description
WID	INT	PRIMARY KEY, AUTO_INC	Unique warehouse identifier
Name	VAR- CHAR(100)	NOT NULL	Facility commercial name
Location	VAR- CHAR(255)	NOT NULL	Physical geographic address
Capacity	INT	NOT NULL	Max storage unit threshold
Temperature_Range	VARCHAR(50)	NULLABLE	Designated operating temperature

7.1.3 Table: product

Field	Type	Constraint	Description
ProductID	INT	PRIMARY KEY, AUTO_INC	Unique product stock ID
Name	VAR- CHAR(100)	NOT NULL	Product item designation
Description	TEXT	NULLABLE	Product storage specification
Price	DECI- MAL(10,2)	NOT NULL	Unit price metric
Temp_Requirement	VARCHAR(50)	NULLABLE	Thermal maintenance range
SupplierID	INT	FK -> supplier(Sup- plierID)	Associated supplier reference

7.1.4 Table: customer_order

Field	Type	Constraint	Description
OrderID	INT	PRIMARY KEY, AUTO_INC	Unique customer purchase order ID

CustomerID	INT	FK -> customer(CustomerID)	Ordering customer reference
Order_Date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Order placement timestamp
Total_Amount	DECIMAL(10,2)	NOT NULL	Gross monetary balance
Status	VARCHAR(50)	DEFAULT 'Pending'	Fulfillment workflow state

7.1.5 Table: inventorylog

Field	Type	Constraint	Description
InventoryLogID	INT	PRIMARY KEY, AUTO_INC	Unique stock log entry ID
ProductID	INT	FK -> product(ProductID)	Referenced product item
WarehouseID	INT	FK -> warehouse(WID)	Destination warehouse ID
Change_Quantity	INT	NOT NULL	Quantity added (+) or removed (-)
Log_Date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Inventory movement timestamp
Note	VARCHAR(255)	NULLABLE	Reason for movement record

8 SQL Commands and Schema

8.1 Data Definition Language (DDL) Script

```

CREATE DATABASE IF NOT EXISTS coldstoredb;
USE coldstoredb;

-- User Table
CREATE TABLE user (
    UserID INT AUTO_INCREMENT PRIMARY KEY,
    Username VARCHAR(50) NOT NULL UNIQUE,
    Password VARCHAR(255) NOT NULL
);

-- Warehouse Table
CREATE TABLE warehouse (
    WID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Location VARCHAR(255) NOT NULL,
    Capacity INT NOT NULL,
    Temperature_Range VARCHAR(50)
);

-- Supplier Table
CREATE TABLE supplier (
    SupplierID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL
);

-- Customer Table
CREATE TABLE customer (
    CustomerID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL
);

-- Product Table
CREATE TABLE product (
    ProductID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Description TEXT,
    Price DECIMAL(10, 2) NOT NULL,
    Temp_Requirement VARCHAR(50),
    SupplierID INT,
    FOREIGN KEY (SupplierID) REFERENCES supplier(SupplierID) ON DELETE SET NULL
);

-- Customer Order Table
CREATE TABLE customer_order (
    OrderID INT AUTO_INCREMENT PRIMARY KEY,
    CustomerID INT NOT NULL,
    Order_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
    Total_Amount DECIMAL(10, 2) NOT NULL,
    Status VARCHAR(50) DEFAULT 'Pending',
    FOREIGN KEY (CustomerID) REFERENCES customer(CustomerID) ON DELETE CASCADE
);

```

```
-- Inventory Log Table
CREATE TABLE inventorylog (
InventoryLogID INT AUTO_INCREMENT PRIMARY KEY,
ProductID INT NOT NULL,
WarehouseID INT NOT NULL,
Change_Quantity INT NOT NULL,
Log_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
Note VARCHAR(255),
FOREIGN KEY (ProductID) REFERENCES product(ProductID) ON DELETE CASCADE,
FOREIGN KEY (WarehouseID) REFERENCES warehouse(WID) ON DELETE CASCADE
);

-- Temperature Sensor Table
CREATE TABLE temperature_sensor (
SensorID INT AUTO_INCREMENT PRIMARY KEY,
WarehouseID INT NOT NULL,
FOREIGN KEY (WarehouseID) REFERENCES warehouse(WID) ON DELETE CASCADE
);

-- Temperature Log Table
CREATE TABLE temperature_log (
LogID INT AUTO_INCREMENT PRIMARY KEY,
SensorID INT NOT NULL,
Recorded_Temp DECIMAL(5, 2) NOT NULL,
Timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (SensorID) REFERENCES temperature_sensor(SensorID) ON DELETE CASCADE
);

-- Feedback Table
CREATE TABLE feedback (
FeedbackID INT AUTO_INCREMENT PRIMARY KEY,
UserID INT NOT NULL,
Comment TEXT,
Rating INT CHECK (Rating BETWEEN 1 AND 5),
Feedback_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (UserID) REFERENCES user(UserID) ON DELETE CASCADE
);
```

8.2 View Definition Scripts

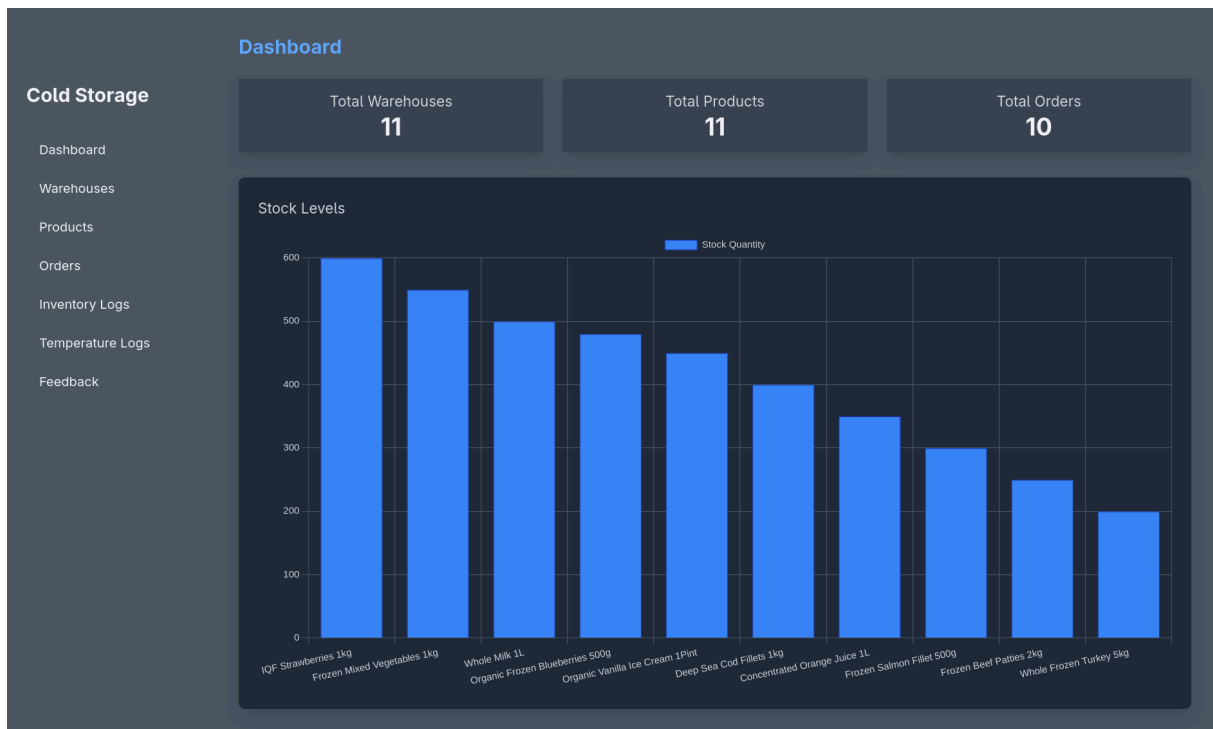
```
-- Active Orders View
CREATE VIEW activeorders AS
SELECT co.OrderID, c.Name AS CustomerName, co.Order_Date, co.Total_Amount, co.Status
FROM customer_order co
JOIN customer c ON co.CustomerID = c.CustomerID
WHERE co.Status NOT IN ('Completed', 'Cancelled');

-- Completed Orders View
CREATE VIEW completedorders AS
SELECT co.OrderID, c.Name AS CustomerName, co.Order_Date, co.Total_Amount, co.Status
FROM customer_order co
JOIN customer c ON co.CustomerID = c.CustomerID
WHERE co.Status = 'Completed';
```

```
-- Stock Summary View
CREATE VIEW stocksummary AS
SELECT
    ROW_NUMBER() OVER (ORDER BY SUM(il.Change_Quantity) DESC) AS StockID,
    p.Name AS Product,
    w.Name AS Warehouse,
    SUM(il.Change_Quantity) AS Quantity
FROM inventorylog il
JOIN product p ON il.ProductID = p.ProductID
JOIN warehouse w ON il.WarehouseID = w.WID
GROUP BY p.ProductID, w.WID;
```

9 UI Design and Interface Structure

The front-end user interface is built with plain HTML and communicates with backend API endpoints using JavaScript's `fetch()` API.



Cold Storage

Dashboard

Warehouses

Products

Orders

Inventory Logs

Temperature Logs

Feedback

Warehouse Management

Add/Edit Warehouse

Name:

Location:

Capacity:

Temperature Range:

ID	Name	Location	Capacity	Temperature Range	Actions
1	North Cold Hub	Chicago, IL	50000	-20°C to 4°C	Edit Delete
2	Pacific Freeze Center	Seattle, WA	35000	-25°C to -10°C	Edit Delete
3	Central Logistics Facility	Dallas, TX	60000	0°C to 10°C	Edit Delete
4	Atlantic Cold Depot	Newark, NJ	45000	-18°C to 2°C	Edit Delete
5	Sunshine State Cold	Miami, FL	40000	-5°C to 5°C	Edit Delete
6	Midwest Chill Store	Minneapolis, MN	55000	-30°C to 0°C	Edit Delete
7	Rocky Mountain Freeze	Denver, CO	30000	-20°C to -5°C	Edit Delete
8	Gulf Coast Storage	Houston, TX	48000	-15°C to 4°C	Edit Delete
9	Northwest Deep Freeze	Portland, OR	38000	-28°C to -12°C	Edit Delete
10	Great Lakes Chiller	Detroit, MI	52000	-22°C to 2°C	Edit Delete
11	Apex Refrigerated Depot	Atlanta, GA	30000	-30C to 15C	Edit Delete

Cold Storage

Inventory Logs

Dashboard

Warehouses

Products

Orders

Inventory Logs

Temperature Logs

Feedback

Search by product or warehouse

Log ID	Product	Warehouse	Quantity Change	Date	Note
1	Whole Milk 1L	North Cold Hub	500	27/9/2026, 4:48:43 am	Initial stock intake
2	Frozen Salmon Fillet 500g	Pacific Freeze Center	300	28/9/2026, 4:48:43 am	Shipment arrival
3	Organic Vanilla Ice Cream 1Pint	North Cold Hub	450	29/9/2026, 4:48:43 am	Batch #A42 received
4	IQF Strawberries 1kg	Central Logistics Facility	600	30/9/2026, 4:48:43 am	Bulk shipment intake
5	Frozen Beef Patties 2kg	Atlantic Cold Depot	250	1/10/2026, 4:48:43 am	Pallet deposit
6	Deep Sea Cod Fillets 1kg	Sunshine State Cold	400	2/10/2026, 4:48:43 am	Import batch arrival
7	Concentrated Orange Juice 1L	Midwest Chill Store	350	3/10/2026, 4:48:43 am	Stock replenishment
8	Frozen Mixed Vegetables 1kg	Rocky Mountain Freeze	550	4/10/2026, 4:48:43 am	Produce delivery
9	Whole Frozen Turkey 5kg	Gulf Coast Storage	200	5/10/2026, 4:48:43 am	Poultry intake
10	Organic Frozen Blueberries 500g	Northwest Deep Freeze	480	6/10/2026, 4:48:43 am	Organic fruit load

Cold Storage

Temperature Logs

Dashboard

Warehouses

Products

Orders

Inventory Logs

Temperature Logs

Feedback

-10

Log ID	Sensor ID	Temperature (°C)	Timestamp	Warehouse
2	2	-19.50 °C	6/10/2026, 7:48:56 pm	Pacific Freeze Center
4	4	-18.20 °C	6/10/2026, 9:48:56 pm	Atlantic Cold Depot
6	6	-28.50 °C	6/10/2026, 11:48:56 pm	Midwest Chill Store
7	7	-15.00 °C	7/10/2026, 12:48:56 am	Rocky Mountain Freeze
8	8	-12.40 °C	7/10/2026, 1:48:56 am	Gulf Coast Storage
9	9	-25.10 °C	7/10/2026, 2:48:56 am	Northwest Deep Freeze
10	10	-20.00 °C	7/10/2026, 3:48:56 am	Great Lakes Chiller

10 Implementation Details

10.1 Technology Stack & Architecture Overview

The backend application is implemented in Python using the Flask web framework, serving both JSON REST endpoints and HTML template routes. Database interactions with the `coldstoredb` schema use the `mysql-connector-python` driver with dictionary cursors to automatically map query rows into JSON-serializable dictionaries.

Layer	Technology	Implementation Function
Backend Core	Flask 3.0 (Python)	Handles HTTP routing, request parsing, session management, and JSON responses.
Database Core	MySQL 8.0 Engine	Stores operational records, executes JOIN queries, and exposes pre-compiled dynamic views.
Database Driver	<code>mysql-connector-python</code>	Provides thread-safe connections with parameterized query execution.
Security	<code>bcrypt</code> & Web Sessions	Manages salted credentials hashing and session state (<code>secret_key = 'srm12345'</code>).
Frontend View	Jinja2 + Bootstrap 5	Renders template layouts (<code>dashboard.html</code> , <code>warehouses.html</code> , <code>products.html</code> , etc.).

10.2 Key Code Snippets

10.2.1 1. Centralized Database Connection Utility

A helper function establishes connection instances with basic exception handling:

```
def get_db_connection():
    try:
        connection = mysql.connector.connect(
            host="localhost",
            user="root",
            password="password",
            database="coldstoredb"
        )
        return connection
    except Error as e:
        print(f"Error connecting to MySQL: {e}")
        return None
```

10.2.2 2. Multi-Method Controller Pattern (Warehouse CRUD)

Handles complete resource management (GET, POST, PUT, DELETE) within a single endpoint using parameterized SQL execution:

```
@app.route('/api/warehouses', methods=['GET', 'POST', 'PUT', 'DELETE'])
def manage_warehouses():
    conn = get_db_connection()
    if conn is None:
        return jsonify({"error": "Database connection failed"}), 500

    cursor = conn.cursor(dictionary=True)
```

```

if request.method == 'GET':
    cursor.execute("SELECT WID, Name, Location, Capacity, Temperature_Range FROM
warehouse")
    warehouses = cursor.fetchall()
    cursor.close()
    conn.close()
    return jsonify(warehouses)

elif request.method == 'POST':
    data = request.json
    cursor.execute("""
        INSERT INTO warehouse (Name, Location, Capacity, Temperature_Range)
        VALUES (%s, %s, %s, %s)
        """, (data['Name'], data['Location'], data['Capacity'],
data['Temperature_Range']))
    conn.commit()
    cursor.close()
    conn.close()
    return jsonify({"message": "Warehouse added"})

# PUT and DELETE follow the same parameterized pattern

```

10.2.3 3. Dynamic Search & Parameterized Querying

Appends dynamic WHERE filters safely using query parameters (%s) to avoid SQL injection:

```

@app.route('/api/inventory_logs', methods=['GET'])
def get_inventory_logs():
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    query = """
        SELECT il.InventoryLogID, p.Name AS Product, w.Name AS Warehouse,
il.Change_Quantity, il.Log_Date, il.Note
        FROM inventorylog il
        JOIN product p ON il.ProductID = p.ProductID
        JOIN warehouse w ON il.WarehouseID = w.WID
        """
    params = []
    if 'search' in request.args:
        query += " WHERE p.Name LIKE %s OR w.Name LIKE %s"
        params = [f"%{request.args['search']}%", f"%{request.args['search']}%"]

    cursor.execute(query, params)
    logs = cursor.fetchall()
    cursor.close()
    conn.close()
    return jsonify(logs)

```

10.2.4 4. Dynamic View Integration (Stock Aggregation & Order Pipeline)

Consumes pre-compiled MySQL views (stocksummary, activeorders, completedorders) directly in API controllers:

```

@app.route('/api/stock_summary', methods=['GET'])
def get_stock_summary():
    conn = get_db_connection()

```

```

cursor = conn.cursor(dictionary=True)
cursor.execute("""
    SELECT StockID, Product, Warehouse, Quantity
    FROM stocksummary
    WHERE Quantity > 0
    ORDER BY Quantity DESC
    LIMIT 10
""")
stock = cursor.fetchall()
cursor.close()
conn.close()
return jsonify(stock)

```

10.2.5 5. Session-Aware User Operations

Retrieves the logged-in user's context directly from session['user_id'] for data isolation:

```

@app.route('/api/feedback', methods=['GET', 'POST'])
def manage_feedback():
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)

    if request.method == 'GET':
        cursor.execute("SELECT FeedbackID, Comment, Rating, Feedback_Date FROM
feedback WHERE UserID = %s", (session['user_id'],))
        feedback = cursor.fetchall()
        cursor.close()
        conn.close()
        return jsonify(feedback)

```

11 Conclusion and Future Enhancements

11.1 Conclusion

The Cold Storage Warehouse Management System (coldstoredb) provides a normalized database architecture paired with a Flask REST API and web dashboard. Bounding data entities to Third Normal Form (3NF) eliminated update anomalies and redundant attributes. SQL views streamline complex queries, while foreign key constraints safeguard system relationships across operations. Parameterized query routines protect against SQL injection vulnerabilities, and session-based access controls secure warehouse telemetry and order management processes.

11.2 Future Enhancements

The modular design of coldstoredb provides a solid foundation for future extensions:

Automated IoT Hardware Integration: Setting up direct WebSockets or MQTT message brokers to stream real-time temperature logs straight from physical warehouse sensors.

Predictive Spoilage Analytics: Applying machine learning algorithms to inventory age and ambient temperature logs to forecast batch expiration risks.

Multi-Factor Authentication (MFA): Enhancing login security with TOTP-based secondary verification steps for administrative user accounts.

Mobile Field Application: Developing a dedicated cross-platform barcode scanning application to simplify warehouse floor inventory intake and dispatch processes.

12 Appendix: GitHub Repository and Setup Instructions

12.1 Source Code Repository

The full source code, SQL database scripts, and project assets are hosted on GitHub:

Repository URL: <https://github.com/NewAccount1729/DBMS-Course-Project>

12.2 Quick-Start Local Deployment Guide

1. Clone the project repository

```
git clone https://github.com/NewAccount1729/DBMS-Course-Project
```

```
cd DBMS-Course-Project
```

2. Configure Python virtual environment

```
python3 -m venv venv
```

```
source venv/bin/activate # On Windows use: venv\Scripts\activate
```

3. Install backend dependency packages

```
pip install flask mysql-connector-python bcrypt
```

4. Provision MySQL Database

```
mysql -u root -p < database_schema_and_seed.sql
```

5. Launch Application Server

```
python app.py
```